ATLAS · PRICING & UNIT ECONOMICS

Cost Model v2

Re-pricing the tier ladder against measured LLM cost

Revision 2 · 13 July 2026 · Reflects prompt caching shipped in v1.3.8

The finding in one line

A run is not a unit of cost. Measured across 598 real runs, run cost ranges from $0.00056 to $0.20636 — a 369× spread — and the Business tier, at its own 5,000-run cap, loses $432/month.

This document sets out the evidence, the options considered, and the tier table we recommend. Every number in it is measured, not modelled: it comes from live stress tests run against the production codebase on 13 July 2026, in which thirteen workflows were built through the conversational builder and executed 598 times against real connectors.

What changed in this revision

Prompt caching shipped to production (v1.3.8) between the first draft and this one. Build cost fell 61%, from $0.569 to $0.2215. Run costs fell 4–24% depending on shape. Every figure below has been re-measured against the shipped code — the run caps are barely affected, but a build now costs one run-unit instead of three, which changes the build allowance materially. Section 8 supersedes the first draft.

1. Executive summary

- The current ladder prices by run count. Run count does not predict cost — payload size does. The two are uncorrelated.
- At its 5,000-run cap, Business costs ~$1,032/mo in LLM spend against $600 of revenue. Team goes negative in the worst case too. Only Solo has real headroom.
- Prompt caching (shipped v1.3.8) cut build cost 61% and web-search runs 24%. It did NOT move the worst-case run, which is what sets the caps: that cost is a volatile payload, and volatile payloads cannot be cached.
- Converger (build) spend is still capped by nothing at all. A $20 Solo tenant is structurally permitted to burn far more than their subscription building workflows without ever touching a run cap.
- We recommend keeping runs as the unit — they are legible and familiar — but sizing each tier so that margin holds even if every run is the most expensive kind. Light users then yield 96%+ margin; the worst possible user still yields ~65%.
- We explicitly reject input truncation as a cost control. On digest-shaped workflows it is not a quality trade-off, it is a correctness bug.

2. How the data was gathered

A clean measurement environment was established and every LLM call was attributed per tenant via llm_cost_log, cross-checked against the workflow_runs.cost_usd ledger. The two agree.

PARAMETER	VALUE
Date	13 July 2026
Tenant	agntic (founding plan, unlimited)
Model	claude-sonnet-4-6 (balanced tier), claude-haiku-4-5 (chat)
Workflows built	13, all via the conversational builder UI
Runs executed	598 real runs (test runs excluded)
Connectors exercised	Slack, Gmail, Calendar, Drive, Sheets, Web search, Web fetch, Inbox
Total spend	$29.12 pre-caching; re-measured post-caching in section 5
Prompt caching	Shipped v1.3.8 mid-study; all headline figures re-measured after

Runs were fired through POST /api/console/workflows/:id/run — the exact endpoint the "Run now" button calls — so they are counted, cost-logged and billable in precisely the way a customer's runs are.

3. The measured cost of a run

Ten workflow shapes, ordered by cost. The right-hand column is the mean input-token payload reaching the model — the variable that actually drives price.

WORKFLOW SHAPE	CONNECTOR	BEFORE	SHIPPED	TOK IN
web_fetch -> extract -> rewrite	Web	$0.21509	**$0.20636**	47,855
calendar + gmail + web -> synthesis	Google/Web	$0.13969	n/m	29,293
web_search -> summarize	Web	$0.10343	**$0.07868**	19,261
drive_list_files -> llm	Google	$0.07212	n/m	22,852
sheets_read -> llm	Google	$0.01311	n/m	1,143
calendar_list_events -> llm	Google	$0.01206	n/m	2,831
slack_history -> summarize	Slack	$0.00738	n/m	1,625
llm -> slack post (write)	Slack	$0.00266	n/m	222
gmail_search -> summarize *	Google	$0.00061	$0.00062	124

WORKFLOW SHAPE	CONNECTOR	BEFORE	SHIPPED	TOK IN
llm -> inbox	—	$0.00055	$0.00056	77

* Measured against an empty inbox. A populated inbox would move this shape materially upward; treat it as a floor, not a forecast. "n/m" = not re-measured post-caching; these shapes have no repeated round-trips, so no material change is expected.

Building a workflow costs a further $0.2215 (was $0.569 before caching) — roughly flat across all shapes, regardless of connector or node count. A build is 20–35 Sonnet calls, because the converger re-sends its full context on every proposal turn. One build now costs about as much as one worst-case run, or roughly 400 trivial runs.

4. The central insight

> **Cost tracks payload size, not workflow complexity**
> drive_list_files -> llm is a two-step workflow a user would describe as "just list my files". It costs $0.07212 per run — 130× the trivial shape and 70% of a full web search — purely because Google Drive returns a 22,852-token payload into the model. Meanwhile a six-node, three-connector workflow with a small payload is cheap. Node count, connector count and step count all fail to predict cost. Only tokens predict cost.

This is why the current ladder cannot be repaired by moving the cap. Any fixed run allowance is simultaneously too stingy for the light user (whose runs cost $0.0006) and financially fatal against the heavy one (whose runs cost 391× more). The unit itself is the defect.

5. Prompt caching (shipped, v1.3.8)

The converger re-sends a byte-stable capability catalog — roughly 7,500 tokens for a tenant with Slack, Google, Airtable, Web and filesystem connected — as its system prompt on every one of its ~21 proposal turns per build. Marking that prefix cacheable bills it at a tenth of the input rate on reads instead of full price. The prompt bytes are unchanged, so the model sees exactly what it saw before: output quality cannot be affected, and the P3 gate reproduces the frozen canonical spec identically before and after.

SURFACE	BEFORE	AFTER	CHANGE
Workflow BUILD	$0.5690	**$0.2215**	**-61%**
Run: web_search -> summarize	$0.10343	$0.07868	-24%
Run: worst case (web_fetch chain)	$0.21509	**$0.20636**	**-4%**
Run: trivial (identical-spec A/B)	$0.00055	$0.00056	noise

> **Caching does not rescue the run economics — and this is the point**
> Runs cache only where a node makes repeated LLM round-trips: the native web-search tool loops internally, so each continuation re-reads the cached prefix (97% cached on that node, -24% on the run). But the WORST run barely moved, because its cost is a 47,855-token article payload — fresh every run, and therefore uncacheable by construction. Since the tier caps are priced against the worst case, caching leaves them essentially unchanged. What it changes is the cost of a BUILD, which fell by 61%.

6. Why the current ladder fails

Worst case below is a tenant running the most expensive measured shape ($0.20636/run, post-caching) to their cap. Breakeven is the run count at which LLM cost alone equals the plan price.

PLAN	PRICE	RUN CAP	BREAKEVEN	HEADROOM	VERDICT
Solo	$20	30	93	3.1×	**safe**
Professional	$50	200	232	1.16×	**razor-thin**
Team	$200	1,000	930	0.93×	**NEGATIVE**
Business	**$600**	**5,000**	2,789	0.56×	**-$475/mo**

A Business tenant running web-fetch briefings to their cap costs $1,032/month against $600 of revenue (it was $1,075 before caching; caching does not save this tier). This is not a tail risk: it is the advertised allowance, used as advertised.

The uncapped build hole

monthlyRuns is enforced only at the run choke point (registerRunBudgetCheck). Builder chat and converger spend are counted against no plan limit whatsoever. The sole backstop is TENANT_DAILY_USD_LIMIT, a flat $25/day applied identically to every plan — which permits a $20/month Solo tenant to consume up to $750/month in build spend while never running a single workflow.

7. Options considered

Option A — Cost-denominated credits

Replace runs with a credit equal to a fixed quantum of underlying LLM cost. Margin becomes structurally guaranteed for any workflow shape, including connectors not yet built. Rejected as the primary model: credits are illegible to buyers, and metered pricing creates purchase anxiety on a product whose whole promise is "just describe what you want".

Option B — Clamp payloads, keep generous caps

Truncate every node input to ~8k tokens before it reaches the model. This bounds the worst run at roughly $0.054 and would let the current caps stand almost unchanged. Rejected on correctness grounds — see below.

> **Why we will not truncate**
> On digest-shaped workflows — "summarize my unread emails", "summarize this channel", "tell me what is in my Drive" — truncation is not a degradation of quality, it is a silent correctness failure. The user asked for a summary of everything and receives a confident summary of some things, with no warning that the rest was discarded. Completeness is the entire value proposition of those workflows. A wrong digest is worse than an expensive one.

Option C — Price the worst case, keep runs (recommended)

Keep the run as the unit, and size each tier so that gross margin holds even if every run is the most expensive kind. The low cap is itself the safety mechanism: it bounds the blast radius of an expensive workflow without ever constraining what that workflow is allowed to do.

8. Recommended tier table

PLAN	PRICE	RUNS/MO	WORKFLOWS	WORST-CASE GM	LIGHT-USER GM
Solo	$20	**30**	1	65%	96%
Professional	$50	**75**	3	66%	96%
Team	$200	**300**	10	66%	97%
Business	$600	**900**	Unlimited	66%	97%

- A workflow build costs 1 run from the allowance. Before caching a build cost 2.6 run-units, which meant a Solo user could afford only two builds a month alongside a daily automation. At 1.07 units they can afford seven. This is the single biggest product effect of the caching work.
- Test runs remain exempt. Users must be able to iterate on a draft without spending their allowance — this is what makes the caps tolerable.
- Prices are unchanged. Only the allowances move.
- Solo is set at 30 rather than 25 so that a weekday-daily automation (22 runs) still leaves room to iterate.

9. Why these numbers

The caps are derived, not chosen. Each is the largest allowance that still yields ~65% gross margin if every single run is the most expensive shape measured ($0.20636), net of Stripe fees at 2.9% + $0.30.

The consequence is a margin floor that holds across the entire range of customer behaviour:

IF THE CUSTOMER'S RUNS ACTUALLY COST…	SOLO	PROF.	TEAM	BUSINESS
$0.0006 (llm -> inbox)	96%	96%	97%	97%
$0.013 (sheets, calendar)	94%	94%	95%	95%
$0.072 (drive listing)	85%	85%	86%	86%
$0.079 (web search, cached)	84%	84%	85%	85%
$0.206 (worst measured)	**65%**	**66%**	**66%**	**66%**
$0.600 (pathological)	5%	7%	7%	7%

> **The safety multiple**
> At these allowances a single run would have to cost roughly $0.64 before any tier turns negative — about 3× more expensive than the worst workflow we could construct across eight connectors. Light users yield 96%+; the worst user we can imagine still yields 65%. This is the property the ladder was supposed to have and did not.

10. Workflow count must match the run budget

The old ladder advertised allowances that its own run budget could not fund. A weekday-daily automation consumes 22 runs per month, so an honest tier must fund at least that many runs per advertised workflow.

PLAN	RUNS	DAILY WORKFLOWS FUNDED	OLD ADVERTISED COUNT	
Solo	25	1	1	**ok**
Professional	75	3	10	**mismatch**
Team	300	13	50	**mismatch**
Business	900	40	Unlimited	**mismatch**

Under the old table, a Professional customer who built the ten workflows they were sold, and scheduled them daily, would exhaust their 200 runs inside week two. The recommended workflow counts are set to what the run budget can actually sustain, so the promise and the allowance agree.

11. Required engineering

A per-run cost circuit breaker

The one genuine hole in an uncapped-cost model is that the worst run is unbounded — $0.215 is merely the worst shape we happened to build, not a ceiling. The fix is a hard stop, not a clamp: if a single run's LLM spend crosses ~$1.00, abort the run and surface the error.

This bounds the tail while never degrading a run that legitimately needs 47k tokens. It is a strictly better failure mode than truncation: a customer whose research workflow is too expensive is told so, and can split it, rather than silently receiving an incomplete answer.

Prompt-cache the converger

93.2% of all tokens consumed in this test were input tokens. The converger re-sends its entire context on every proposal turn, which is why a build costs $0.57. Anthropic prompt caching bills cache reads at roughly a tenth of base input. This is the single largest available cost reduction and it does not touch quality.

Make the daily USD ceiling per-plan

TENANT_DAILY_USD_LIMIT is a flat $25/day for every tenant regardless of plan. It should scale with the plan it is protecting.

12. Defects found during the study

DEFECT	DETAIL

DEFECT	DETAIL
{{today}} template leak	The converger emits timeMin: "{{today}}" into the Google Calendar API. The engine does not resolve {{today}}, so the literal string is sent to Google and rejected with 400 Bad Request. Confirmed: the identical step succeeds when instructed to leave timeMin at its default.
Converger drops requested steps	Asked for a note delivered to Slack AND emailed via Gmail, and confirmed twice, the ratified spec contained only the Slack delivery. The Gmail step was silently omitted — no warning, and the workflow published as if complete.
Dead model field	The converger sets "model":"claude-opus-4-5" on LLM nodes. src/workflows/node-types/llm.js never reads config.model, so it is ignored today — but it is an Opus-priced cost bomb the moment anyone wires it up.
Airtable is not discoverable	The capability catalog exposes no list_bases. Atlas cannot enumerate a tenant's Airtable bases, so airtable_list_records can only ever be configured with a base ID the user pastes by hand — which breaks the conversational promise for that connector.
web_fetch approaches timeout	The web_fetch -> extract -> rewrite shape has a median run duration of 113s against the 180s connector timeout. This shape will begin failing on slower pages.

13. Open questions

- The gmail_search shape was measured against an empty inbox (124 tokens). A populated inbox will move it upward. It cannot threaten the margin floor — the run cap bounds that — but it does affect how generous 25 runs feels to a real Solo customer. Worth measuring.
- Prompt caching should be implemented and the build cost re-measured before the build-costs-3-runs rule is finalised. If caching lands, a build may be cheap enough to make free again.
- These figures are Sonnet-based. Routing summarisation steps to Haiku would cut the mid-range shapes roughly four-fold and is worth testing against output quality.